

§11.4.170 Host-Render Visual Proof — ota-manager LoginPage (first increment)

Revision: 1 **Last modified:** 2026-07-09T00:00:00Z **Scope:** clients/ota-manager frontend only (harness added under clients/ota-manager/visual/). **Mandate:** §11.4.170 — device-independent HOST-side rendered-pixel visual proof, per screen×state×{light,dark}, dual-validated by (i) golden image-diff AND (ii) an OCR/vision layout oracle.

1. What was stood up

The FIRST provable increment of a §11.4.170 host-render harness for ota-manager: the **real LoginPage component** (src/features/auth/login-page.tsx, imported — not copied) is rendered to real PNG files ON THE HOST by headless Chromium via Playwright, with **no dev server, no emulator, no running app**. The component is mounted with the exact providers it depends on (QueryClientProvider + TanStack-Router memory history — LoginPage → useLogin → useMutation/useNavigate).

Harness stack (installed, versions captured):

Tool	Version	Role
Node	v24.18.0	runtime
Playwright (core)	1.55.1	host render → PNG + real rendered bounding boxes
Chromium (playwright build)	v1193 (140.0.7339.186)	rendering engine — ~/.cache/ms-playwright/chromium-1193
Vite	6.4.3	builds the harness to a static bundle (visual/.out)

Tool	Version	Role
@vitejs/plugin-react + @tailwindcss/vite	(project devDeps)	real React 19 + Tailwind v4 pipeline
tailwindcss	4.3.1	real token/utility CSS from <code>src/index.css</code>
pixelmatch	6.0.0	oracle (i) golden image-diff
pngjs	7.0.0	PNG decode/encode
tesseract	5.3.0 (system binary)	oracle (ii) OCR text- reading

Install evidence: `pnpm install + pnpm add -D playwright@1.55.1 pixelmatch@6.0.0 pngjs@7.0.0` both succeeded; `pnpm exec playwright install chromium` downloaded the matching browser (the cached chromium-1228 did not match the pinned 1.55.1, which needs chromium-1193 — honest version reconciliation, no fake). Chromium launch was verified before use.

2. Screens × states rendered

Representative screen: **LoginPage** (title “OTA Manager”, description, Email + Password fields, “Sign in” button) — in BOTH themes, driven exactly by the `src/index.css` token contract:

- **light** → `document.documentElement.classList = "light"` (light HSL token overrides)
- **dark** → `document.documentElement.classList = "dark"` (tokens fall back to `:root = dark palette`)

Baseline (golden) PNGs, committed:

- `baselines/login-light.png` — 1000×720, real light-theme render
- `baselines/login-dark.png` — 1000×720, real dark-theme render

Both were visually confirmed as genuine, correctly-themed, distinct renders of the real login screen (not blank).

3. Dual-oracle result (both themes PASS)

From `results.json / run.log` (re-run deterministic, exit 0):

theme	(i) image-diff golden-good	(i) image-diff golden-bad	(ii) OCR labels	(ii) layout baseline	(ii) layout golden-bad
light	0.0000% → PASS (matches baseline)	0.7136% → FLAGGED	ALL PRESENT	OK	FLAGGED collapsed submit (398.0×0.0)
dark	0.0000% → PASS (matches baseline)	1.4653% → FLAGGED	ALL PRESENT	OK	FLAGGED collapsed submit (398.0×0.0)

Oracle (i) — golden image-diff (`oracle-diff.mjs`, `pixelmatch`): re-render vs committed baseline differs by 0.0000% (identical → no false positive). A deliberately-mutated render (golden-bad: the submit button collapsed to height 0 — the §11.4.170 forensic “broken button while token tests stay green” case) differs by 0.71%/1.47% and is correctly **FLAGGED**. Diff visualizations written to `diff/login-{light,dark}-{good,bad}-diff.png`.

Oracle (ii) — OCR + rendered-bounds layout (`oracle-ocr.mjs`): Tesseract read all expected labels (“OTA Manager”, “Email”, “Password”, “Sign in”) from BOTH baseline PNGs (`ocr/login-{light,dark}.txt`). The layout oracle over the REAL Playwright bounding boxes confirmed no element is collapsed / clipped / off-screen / overlapping on the baseline, and correctly **FLAGGED** the collapsed submit button on the golden-bad render (`bounds/login-{theme}-{good,bad}.json`).

4. Analyzer self-validation (§11.4.107(10) golden-good + golden-bad)

Neither oracle can silently bluff — each was proven to PASS its golden-good input AND FAIL/flag its golden-bad input:

- `image_diff_analyzer_sound`: true (good=0.0000% pass, bad≥0.2% flagged, both themes)
- `layout_analyzer_sound`: true (baseline clean, golden-bad collapse detected, both themes)

This is the concrete rebuttal to a value/token-equality test being the sole proof (§11.4.170 forbids that): the collapsed-button regression is invisible to a hex/sp/dp equality unit test but is caught by BOTH rendered-pixel oracles here.

5. Reproduction

From `clients/ota-manager/`:

```
pnpm install
pnpm add -D playwright@1.55.1 pixelmatch@6.0.0 pngjs@7.0.0    #
first time only
pnpm exec playwright install chromium                          #
first time only
pnpm hostrender          # build harness + render + dual-oracle
self-validation
```

Exit 0 = all gates + both analyzer self-validations PASS. Full log: `run.log`.

6. Evidence file manifest (this directory)

- `results.json` — structured machine-readable verdicts (source of truth)
- `run.log` — full pnpm `hostrender` console output (exit 0)
- `baselines/login-{light,dark}.png` — committed golden baselines
- `rerender/login-{light,dark}.png` — identical re-render (golden-good input)
- `mutated/login-{light,dark}-bad.png` — mutated render (golden-bad input)
- `diff/login-{light,dark}-{good,bad}-diff.png` — pixelmatch diff visualizations
- `bounds/login-{light,dark}-{good,bad}.json` — real rendered bounding boxes
- `ocr/login-{light,dark}.txt` — Tesseract OCR output per baseline
- `harness-src/*` — copy of the harness source for reproducibility

Harness source of record lives in-tree at `clients/ota-manager/visual/` (`harness.{html,tsx,css}`, `vite.harness.config.ts`, `lib-render.mjs`, `oracle-diff.mjs`, `oracle-ocr.mjs`, `run-all.mjs`) + pnpm `hostrender` script.

7. Honest findings & scope boundaries (\$11.4.6)

- **Theming-wiring gap (real finding, FACT — not fixed here):** the `ui-store` holds a theme value and `topbar.tsx` toggles it, but **no code applies a `.dark/.light` class to `document.documentElement/body`** (verified: the only match across `src/` is the `@custom-variant dark` declaration in `index.css`). At runtime the toggle changes only the Sun/Moon icon, not the actual palette — `:root (dark)` is always in effect. The harness therefore drives the theme the way `index.css` *defines* it (`light=.light`, `dark=:root`), proving both token sets render correctly; wiring the toggle to a DOM class is a separate app fix (out of scope for this first increment).
- This increment covers ONE screen in its default (empty-form) state. Error/validation/ loading states and other screens (dashboard, devices, releases, ...) are follow-on §11.4.170 coverage, not claimed here.
- The `data-theme` attribute the artifact/theme-toggle convention uses elsewhere is not part of this app; the app's actual contract is the `.light/:root` class split above, which is what was reproduced. No behaviour is claimed that was not rendered and captured.